

Discovery Executive Summary

Project: Discovery-10 · Generated: 24/07/2026, 17:04:58

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), Direct SQL/ORM in Controllers (H6), Domain Boundary Violations (H8), Shared Database Coupling (H9), and Dual-API Logic Duplication (H10).

EXECUTIVE SUMMARY

Klearcom is a monolithic voice/telecom QA platform with three runtime layers: Laravel 12 API (`backend/`), React 19 SPA (`frontend/`), and a parallel Node Express API (`dev-api/`) used for local development. Layers covered: **backend** Laravel (17 `app/` PHP sources), ~~frontend~~ React (15 `src/` TS/TSX/JSX sources), **dev-api** Node (6 JS sources). Architecture intent (Discovery/Connect modules, services, DI) is documented but not enforced — module folders hold only `AGENTS.md`, controllers own Eloquent queries and KPI math, and there is no repository layer. The dominant risks are **missing service/repository boundaries**, **cross-domain Dashboard/Legacy coupling**, and **dual-backend logic duplication** (Laravel → Express), which amplify change cost whenever reachability, IVR tree, or KPI formulas evolve. Frontend is healthier on LOC but still hard-codes API paths in pages and retains legacy class/error-handling patterns without Error Boundaries.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Ra
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	~76 LOC (6 Laravel Api controllers)	
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 handler methods (13 Laravel + 12 Express)	
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	~37 Eloquent/store/Mongo access sites outside repositories	
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	3 (LegacyDataMapper , store.buildTree , duplicated reachability helpers)	
H6	Direct SQL in Controllers	ORM compliance % (queries outside controllers)	>90%	60–90%	<60%	~38%	
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest ~`dev-api/src/server.js` ≈290 LOC)	
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8+ (Dashboard, Legacy, RealTimeTestService, LegacyDashboardWidget, Express KPIs)	
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	80% (4/5 MariaDB business tables read cross-domain)	
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	~87 LOC (9 page/component files)	
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 components with inline api.* paths	
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest ConnectPage.tsx ≈222 LOC)	
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 (small Zustand uiStore)	
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (LegacyMonitorPoller.jsx class + LegacyDashboardWidget.tsx	

						uncaught throw) + no Error Boundaries	
H10	Dual-API Logic Duplication (additional)	Duplicated workflows Laravel ↔ Express (Good 0 · Moderate 1–3 · High Risk >3)	0	1–3	>3	≥5 (KPIs, buildTree, reachability, Discovery CRUD, Connect CRUD)	
H11	Service Locator Abuse (additional)	<code>app()</code> / container resolve call sites in controllers (Good 0 · Moderate 1–3 · High Risk >3)	0	1–3	>3	2 (<code>DiscoveryController</code> , <code>ConnectController</code> dispatch closures)	

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 Missing Service Layer	Extract <code>DashboardService</code> , <code>DiscoveryApplicationService</code> , <code>ConnectApplicationService</code> ; move KPI/tree/reachability workflows out of controllers and Express handlers	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces + Eloquent/in-memory implementations; stop static Eloquent/ <code>store</code> access from controllers	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Replace <code>LegacyDataMapper</code> extract helpers; centralize <code>IvrTreeBuilder</code> + <code>ReachabilityCalculator</code> domain services	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Relocate all Eloquent/Mongo query construction behind repositories; target >90% queries outside HTTP handlers	HIGH RISK	CRITICAL
H8 Domain Boundary Violations	Enforce real code in <code>Modules/Discovery</code> & <code>Modules/Connect</code> ; Dashboard/Legacy consume published APIs only	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Assign table/collection ownership; cross-domain reads via ACL/read models, not direct model imports	HIGH RISK	CRITICAL
F5 Legacy / Inconsistent Component Patterns	Convert <code>LegacyMonitorPoller</code> to a hook with cleanup; add Error Boundaries; quarantine/remove legacy widget; optionally add module API clients for path centralization	MODERATE	MEDIUM
H10 Dual-API Logic Duplication	Collapse Express domain logic into proxy-to-Laravel or shared contracts; stop dual maintenance of KPIs/tree/reachability	HIGH RISK	CRITICAL
H11 Service Locator Abuse	Replace <code>app(RealTimeTestService::class)</code> in dispatch closures with injected service or queued Job DI	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers and Express handlers become thin HTTP adapters; Discovery/Connect workflows are testable via Application Services without booting the full HTTP stack.

- Repository interfaces isolate MariaDB/Mongo persistence, enabling in-memory fakes and eventual schema ownership per bounded context.
- Dashboard and Legacy report through anti-corruption/read-model APIs, so Connect schema changes no longer silently break Discovery reporting.
- A single source of truth for KPI, IVR tree, and reachability formulas eliminates Laravel ↔ Express drift.
- Frontend module API clients plus Error Boundaries reduce endpoint churn and make legacy class/error patterns fail safely.